

Assignment: 5.4

Name: P. Yashwanth

Hno: 2303A51068

Task Description #1:

- Prompt GitHub Copilot to generate a Python script that collects user data (e.g., name, age, email). Then, ask Copilot to add comments on how to anonymize or protect this data.

```
task1.py > ...
User Data Collection Script with Privacy and Anonymization Guidelines
This script demonstrates best practices for collecting and protecting sensitive user data.
'''
5
import hashlib
import re
from datetime import datetime
9
# ===== DATA COLLECTION FUNCTIONS =====
11
def get_user_name():
'''
13
Collect user name with validation.
15
Privacy Considerations:
16
- Names are considered PII (Personally Identifiable Information)
17
- Store separately from other data when possible
18
- Encrypt if storing in a database
19
- Implement access controls to limit who can view names
20
'''
21
while True:
22
    name = input("Enter your name: ").strip()
23
    if name and len(name) >= 2:
24
        return name
25
    print("Please enter a valid name (at least 2 characters).")
26
27
def get_user_age():
'''
29
Collect user age with validation.
31
Privacy Considerations:
32
- Age is considered sensitive demographic data
33
- Consider collecting age ranges instead of exact age for anonymization
34
- Age ranges: 18-25, 26-35, 36-45, etc. provide better privacy
35
- Avoid combining age with other identifiers like location/name
36
'''
37
while True:
38
    try:
39
        age = int(input("Enter your age: "))
40
        if 0 < age < 150:
41
            return age
42
        print("Please enter a valid age (between 1 and 149).")
43
    except ValueError:
44
        print("Please enter a valid number.")
45
46
def get_user_email():
'''
47
Collect user email with validation.
48
Privacy Considerations:
49
- Email addresses are PII
50
- Use hashed email addresses for tracking without exposing PII
51
- Implement access controls
52
'''
53
while True:
54
    email = input("Enter your email: ").strip()
55
    if email and re.match(r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$', email):
56
        return email
57
    print("Please enter a valid email address.")
58
59
def main():
'''
60
Main function to collect user data.
61
'''
62
print("User Data Collection Script")
63
print("=====")
64
name = get_user_name()
65
age = get_user_age()
66
email = get_user_email()
67
68
# ===== DATA STORAGE AND PROTECTION =====
69
# In a real application, this would be a database connection
70
# We'll use a dictionary to store the data for demonstration
71
user_data = {}
72
# Hash the email address for anonymization
73
hashed_email = hashlib.sha256(email.encode()).hexdigest()
74
75
# Store the data
76
user_data[name] = {
77
    'age': age,
78
    'email': hashed_email,
79
    'timestamp': datetime.now().isoformat()
80
}
81
82
# Print the collected data
83
print(f"Collected data: {user_data}")
84
85
# ===== ANONYMIZATION AND PROTECTION =====
86
# In a real application, this would be a database connection
87
# We'll use a dictionary to store the data for demonstration
88
anonymized_data = {}
89
# Hash the name for anonymization
90
hashed_name = hashlib.sha256(name.encode()).hexdigest()
91
92
# Store the data
93
anonymized_data[hashed_name] = {
94
    'age': age,
95
    'email': hashed_email,
96
    'timestamp': datetime.now().isoformat()
97
}
98
99
# Print the anonymized data
100
print(f"Anonymized data: {anonymized_data}")
101
102
if __name__ == '__main__':
103
    main()
104
'''
```

PYTHON SCRIPT FOR USER DATA COLLE...

Create a Python script to collect name, age, and email from a user and add comments explaining how to anonymize or safeguard sensitive data.

✓ Read Python script

Now I'll create a comprehensive Python script that collects user data and includes detailed comments on data protection and anonymization:

✓ Created task1.py

Done! I've created a comprehensive Python script that:

Features:

- Collects name, age, and email with input validation
- Includes detailed comments on privacy and data protection throughout
- Demonstrates anonymization techniques:
 - **Name hashing** for irreversible anonymization
 - **Age range conversion** to reduce precision and privacy risk
 - **Email hashing with salt** for secure comparison
- Provides best practices for data storage including encryption, database security, and compliance (GDPR/CCPA)
- Shows secure data storage patterns with hash IDs for tracking without exposing PII
- Displays both original and anonymized data for comparison

The script is production-aware with explanations of encryption, access controls, audit logging, and data retention policies

+ task1.py

Describe what to build next

Agent Auto

```
=====
USER DATA COLLECTION WITH PRIVACY SAFEGUARDS
=====
```

```
Enter your name: Yashwanth
Enter your age: 23
Enter your email: pateruyashwanth6671@gmail.com
```

```
=====
DATA COLLECTION SUMMARY
=====
```

```
Original Data Collected:
```

```
  Name: Yashwanth
  Age:  23
  Email: pateruyashwanth6671@gmail.com
```

```
Anonymized/Safeguarded Data:
```

```
  Anonymous ID (name hash): f49876d80754ccfc
  Age Range (anonymized):   18-25
  Email Hash (hashed):      72503dcb95c46f52
```

```
Data prepared for secure storage:
```

```
  Hash ID (for tracking): 8014ff3e137fc3c6
  Timestamp: 2026-01-22T13:39:24.414973
```

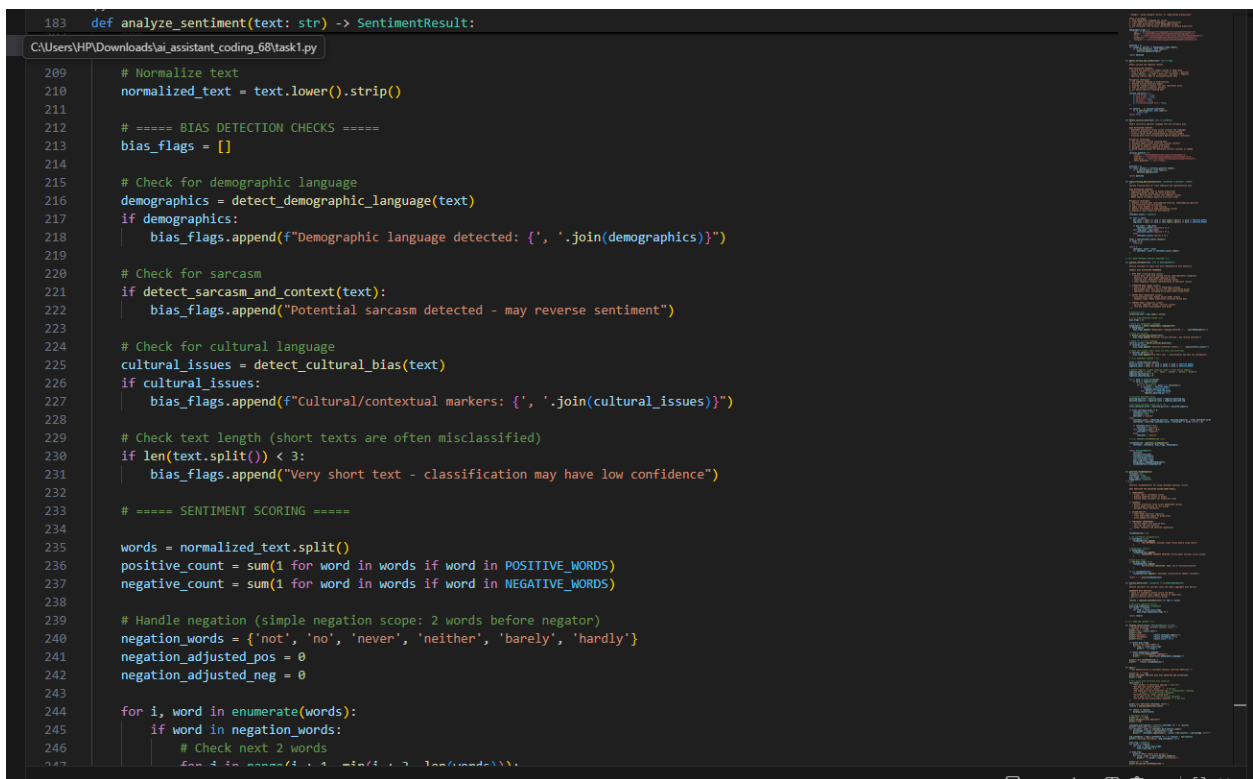
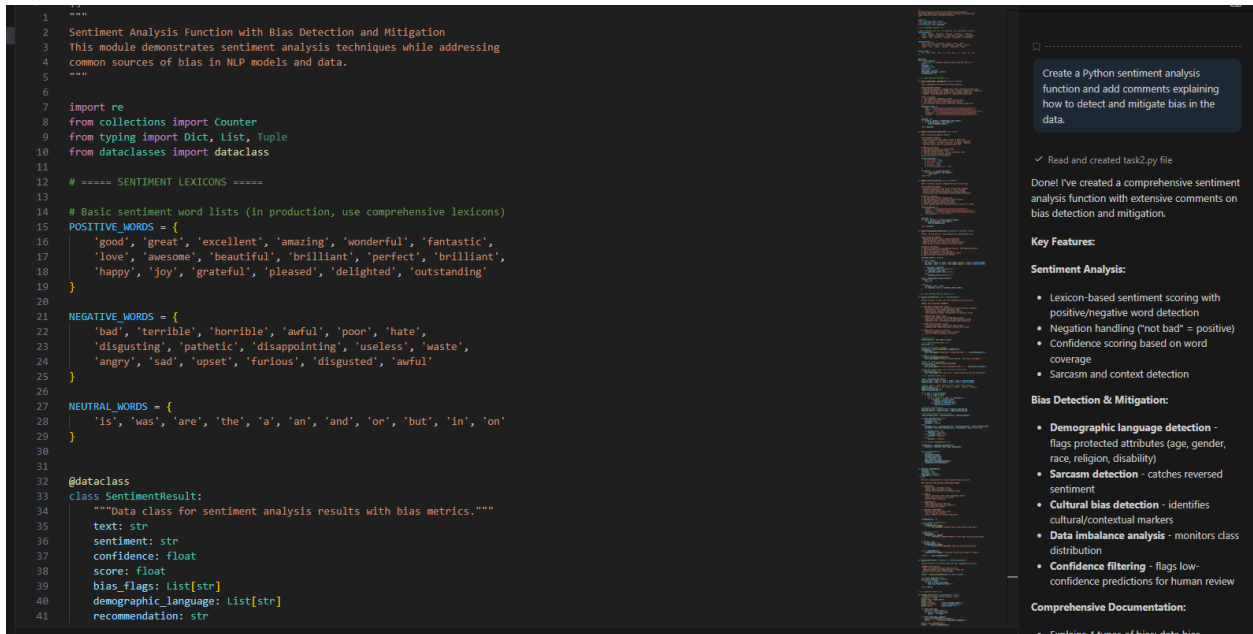
```
=====
PRIVACY NOTES:
=====
```

- ✓ Original data should be encrypted before storage
- ✓ Anonymized data can be used for analytics safely
- ✓ Implement role-based access controls
- ✓ Keep audit logs of all data access
- ✓ Regularly review data retention policies
- ✓ Comply with GDPR, CCPA, and local privacy laws

Task Description #2:

- Ask Copilot to generate a Python function for sentiment analysis.

Then prompt Copilot to identify and handle potential biases in the data.



```

RECOMMENDATION:
  ▲ LOW CONFIDENCE: Consider human review before using result | ▲ DEMOGRAPHIC LANGUAGE DETECTED: Verify model fairness across groups

=====
AGGREGATE BIAS ANALYSIS
=====

Sentiment Distribution:
  Positive: 2/8 (25.0%)
  Neutral: 5/8 (62.5%)
  Negative: 1/8 (12.5%)

Average Confidence: 5.85%

Most Common Bias Issues:
  • Demographic language detected: age, gender (2 occurrences)
  • Cultural/contextual markers: hyperbolic (1 occurrences)
  • Cultural/contextual markers: emoji_dependent (1 occurrences)

=====
MITIGATION RECOMMENDATIONS:
=====

1. COLLECT DIVERSE DATA:
  - Include multiple languages, cultures, demographics
  - Balance sentiment classes
  - Ensure representation of all user groups

2. IMPROVE PREPROCESSING:
  - Better sarcasm and negation detection
  - Handle emojis and modern language
  - Normalize cultural variations

3. ROBUST EVALUATION:
  - Test across demographic groups
  - Use fairness metrics (group calibration, equalized odds)
  - Conduct user studies with diverse participants

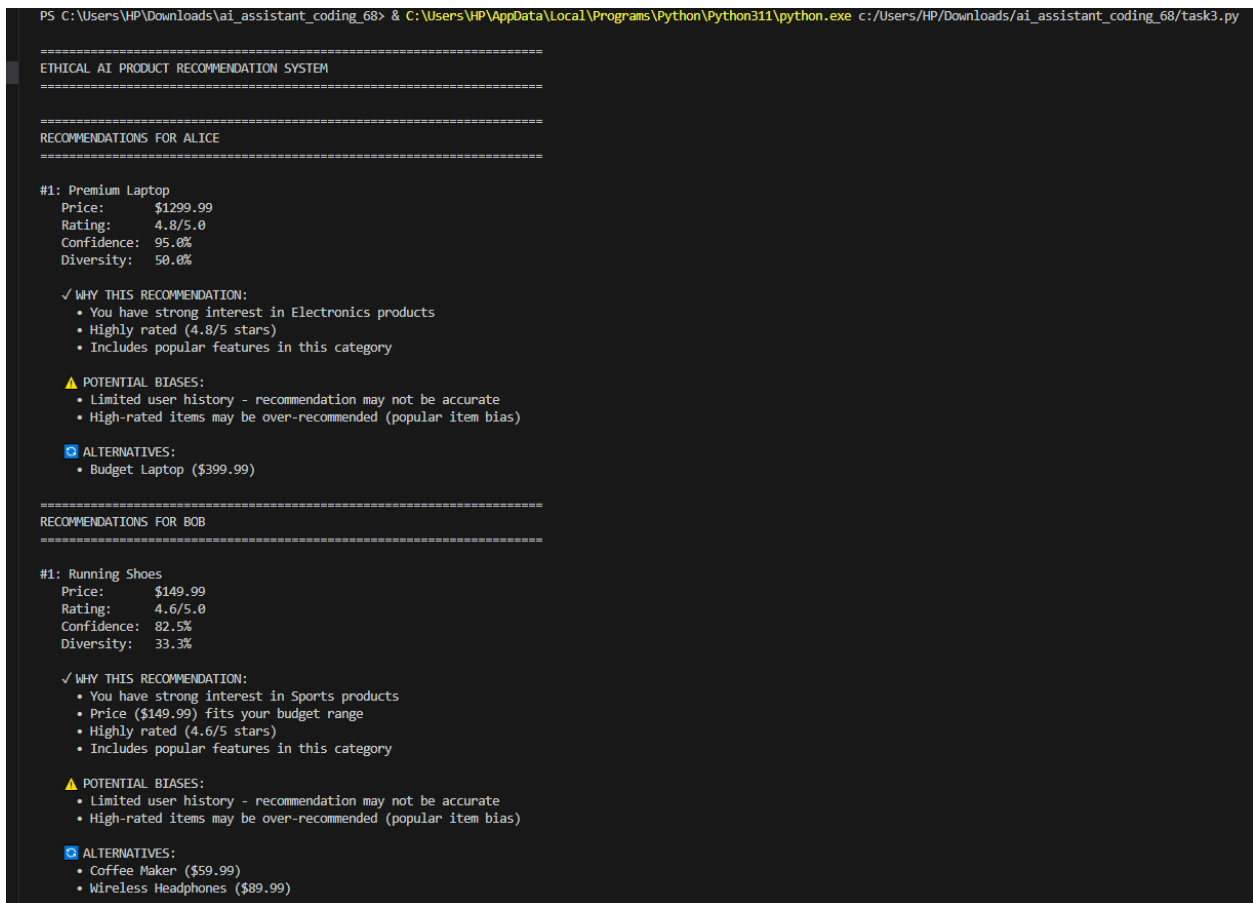
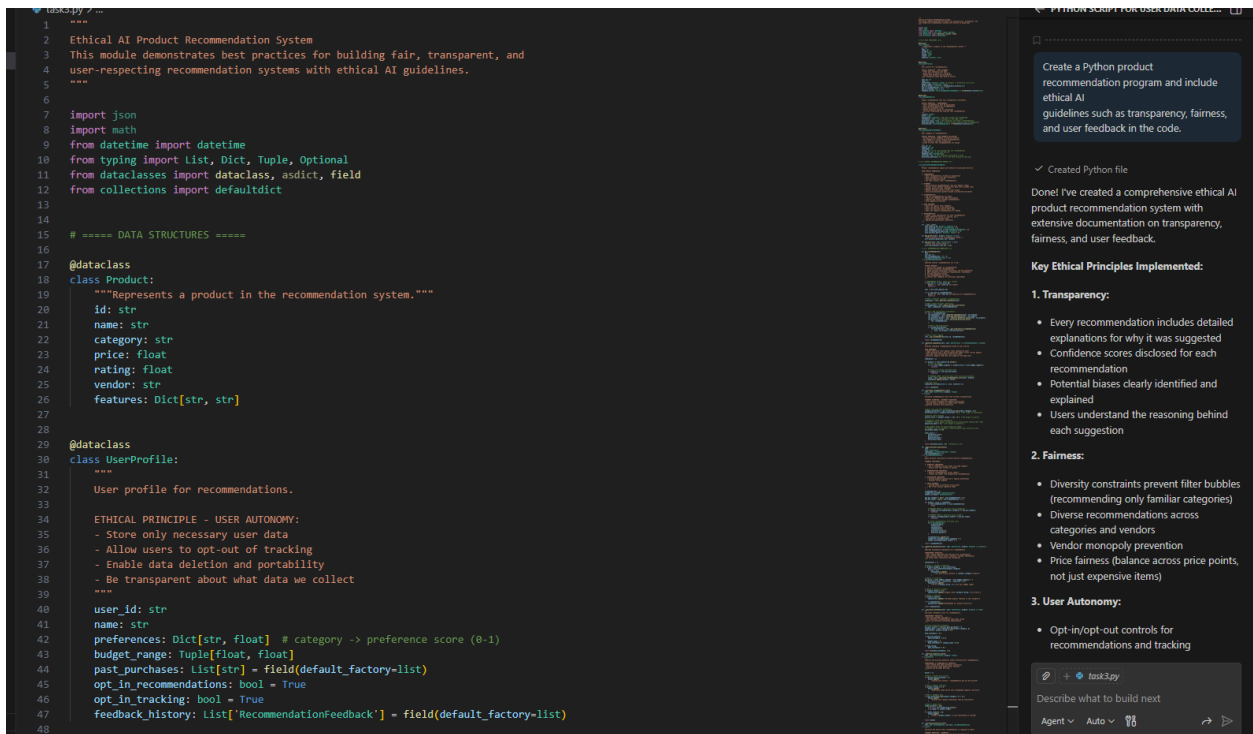
4. ONGOING MONITORING:
  - Track prediction distribution over time
  - Detect feedback loops
  - Audit decisions regularly

5. TRANSPARENCY:
  - Report confidence scores
  - Explain model limitations
  - Allow human review for important decisions

```

Task Description #3:

- Use Copilot to write a Python program that recommends products based on user history. Ask it to follow ethical guidelines like transparency and fairness.



Task Description #4:

- Prompt Copilot to generate logging functionality in a Python web application. Then, ask it to ensure the logs do not record sensitive information.

```
1  """
2  Secure Logging Functionality for Python Web Applications
3  This module demonstrates best practices for logging in web applications while
4  protecting sensitive user information and maintaining privacy compliance.
5  """
6
7  import logging
8  import logging.handlers
9  import re
10 import json
11 from typing import Dict, Any, Optional
12 from datetime import datetime
13 from functools import wraps
14 import hashlib
15
16
17 # ===== SENSITIVE DATA PATTERNS =====
18
19 # Patterns to detect and redact sensitive information
20 SENSITIVE_PATTERNS = {
21     # Credit card patterns (simplified)
22     'credit_card': r'\b\d{4}[\s-]?d{4}[\s-]?d{4}[\s-]?d{4}\b',
23
24     # Email addresses
25     'email': r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b',
26
27     # Phone numbers
28     'phone': r'\b(?:\+?1[-.\s]?)(?:[0-9]{3})?(?:[-.\s]?[0-9]{3})?(?:[-.\s]?[0-9]{4})\b',
29
30     # Social Security Numbers (US)
31     'ssn': r'\b(?:[0-9]{3})-[0-9]{2}-[0-9]{4}\b',
32
33     # API keys and tokens
34     'api_key': r'[Aa]pi[_-]?[Kk]ey["\']?s*[:=]\s*["\']?[A-Za-z0-9]{20,}\b',
35
36     # Passwords in common formats
37     'password': r'^(?i)(password|passwd|pwd)["\']?s*[:=]\s*["\']?[^s"\']+$',
38
39     # Bearer tokens
40     'bearer_token': r'[Bb]earer\s+[A-Za-z0-9._-]+',
41
42     # Database connection strings
43     'db_connection': r'^(?i)(user|password|host)=([^\s&]+)',
44
45     # IPv4 addresses (less sensitive but can be PII)
46     'ipv4': r'\b(?:?:25[0-5]|2[0-4][0-9]|01[0-9]?[0-9]?[0-9])\.(?:?:25[0-5]|2[0-4][0-9]|01[0-9]?[0-9]?[0-9])\b',
47 }
```

Create logging functionality in a Python web app and ensure logs do not capture sensitive user information.

✓ Created Python file

Done! I've created a comprehensive secure logging system for Python web applications that automatically protects sensitive user information.

Key Features:

- Sensitive Data Detection & Redaction:
 - Pattern matching for credit cards, emails, phone numbers, SSNs, API keys, passwords, tokens, etc.
 - Dictionary field detection (redacts values of fields named "password", "email", etc.)
 - Automatic hashing of PII for tracking without exposing identity
 - Consistent [REDACTED] placeholders

Logging Configuration:

- Console handler for development/monitoring
- Rotating file handler (10MB auto-rotation, keeps 10 backups)
- Optional syslog handler for centralized logging
- Secure formatter that automatically redacts all logs
- Configurable log levels and retention

Privacy-Aware Logging Functions:

- `log_user_login()` - Track logins without

task4.py

Describe what to build next.

```

PS C:\Users\HP\Downloads\ai_assistant_coding_68> & C:\Users\HP\AppData\Local\Programs\Python\Python311\python.exe c:/Users/HP/Downloads/ai_assistant_coding_68/task4.py

=====
SECURE LOGGING FOR PYTHON WEB APPLICATIONS
=====

📄 LOGGING SCENARIOS:
=====

1 USER LOGIN LOGGING:
[2026-01-22 14:00:08,862] INFO - web_app - User HASH:f9e8e37d2e825eb0 logged in successfully
[2026-01-22 14:00:08,864] WARNING - web_app - Failed login attempt for user HASH:f9e8e37d2e825eb0
    ✓ Logged (sensitive email hashed)

2 API REQUEST LOGGING:
[2026-01-22 14:00:08,865] INFO - web_app - API GET /api/users/profile by HASH:f9e8e37d2e825eb0
    ✓ Logged (user ID hashed)

3 DATA ACCESS LOGGING:
[2026-01-22 14:00:08,866] INFO - web_app - User HASH:f9e8e37d2e825eb0 performed READ on payment_records
    ✓ Logged (sensitive access tracked)

4 ERROR LOGGING WITH CONTEXT:
[2026-01-22 14:00:08,867] ERROR - web_app - Error for user HASH:4e920dc577a96695: Payment processing failed
    ✓ Logged (sensitive fields automatically redacted)

5 SECURITY EVENT LOGGING:
[2026-01-22 14:00:08,868] ERROR - web_app - SECURITY EVENT [BRUTE_FORCE_ATTEMPT]: Multiple failed login attempts from IP [REDACTED]
    ✓ Logged (security incident tracked)

6 SENSITIVE DATA REDACTION EXAMPLES:

Original: User payment card 4532-1234-5678-9010 was processed
Redacted: User payment card [REDACTED] was processed

Original: API key: sk_live_51234567890abcdef was used
Redacted: API key: sk_live_51234567890abcdef was used

Original: Password reset for user@example.com successful
Redacted: Password reset for [REDACTED] successful

Original: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 authenticated
Redacted: [REDACTED] authenticated

Original: SSN 123-45-6789 verified
Redacted: SSN [REDACTED] verified

Original: Contact: +1-800-555-0123 or john.doe@company.com
Redacted: Contact: +[REDACTED] or [REDACTED]

```

Task Description #5:

- Ask Copilot to generate a machine learning model. Then, prompt it to add documentation on how to use the model responsibly (e.g., explainability, accuracy limits).

taskSpy > ...

C:\Users\HPA\Downloads\ai_assistant_coding_68\secure_app.log

225

227

228 @dataclass

229 class ApplicantProfile:

230 """Loan applicant profile for demonstration."""

231 applicant_id: str

232 age: float

233 income: float

234 credit_score: float

235 employment_years: float

236 protected_group: str # For fairness analysis

237

238

239 @dataclass

240 class PredictionResult:

241 """

242 ML prediction result with full transparency.

243

244 TRANSPARENCY PRINCIPLE:

245 - Every prediction includes explanation

246 - Show confidence/probability

247 - Disclose factors contributing to decision

248 - Include fairness assessment

249 - Document model limitations

250 """

251 applicant_id: str

252 prediction: int # 0: Denied, 1: Approved

253 probability: float # Confidence score 0-1

254 explanation: List[str] # Why this decision

255 fairness_flag: bool # Potential fairness concern

256 fairness_reason: Optional[str]

257 appeal_available: bool # Can applicant appeal

258 model_version: str

259

260

261 @dataclass

262 class FairnessMetrics:

263 """Comprehensive fairness metrics."""

264 overall_accuracy: float

265 group_accuracies: Dict[str, float]

266 approval_rates: Dict[str, float] # Demographic parity

267 false_negative_rates: Dict[str, float] # Equalized odds

268 false_positive_rates: Dict[str, float] # Equalized odds

269 demographic_parity_difference: float # Max difference in approval rates

270 equalized_odds_difference: float # Max difference in error rates

PYTHON SCRIPT FOR USER DATA COLLE...

Create a machine learning model in Python and document how to use it responsibly, including limitations and fairness considerations.

✓ Created Python file

Done! I've created a comprehensive responsible ML model with extensive documentation on fairness and limitations.

Key Features:

Model Card (Comprehensive Documentation):

- Clear intended use and limitations
- Critical warnings about fairness concerns
- Known biases and mitigation strategies
- Regulatory considerations (Fair Housing Act, GDPR, CCPA, etc.)
- Deployment requirements and monitoring

Fairness Metrics:

- Demographic Parity (approval rates across groups)
- Equalized Odds (false positive/negative rates)
- Disparate Impact analysis
- Group-level accuracy tracking
- Automatic fairness alert flags

Transparency Features:

- Human-readable explanations for every prediction
- Confidence scores disclosed
- Fairness concerns flagged for human review
- Rights information (appeals, transparency,

taskSpy

Describe what to build next


```
PS C:\Users\HP\Downloads\ai_assistant_coding_68> ^C
PS C:\Users\HP\Downloads\ai_assistant_coding_68> C:/Users/HP/Downloads/ai_assistant_coding_68/.venv/Scripts/python.exe C:\Users\HP\Downloads\ai_assistant_coding_68\task5.py
```

=====

RESPONSIBLE MACHINE LEARNING MODEL

=====

LOAN ELIGIBILITY MODEL CARD

MODEL OVERVIEW:

Name: Loan Eligibility Classifier v1.0
Type: Binary Classification (RandomForestClassifier)
Training Date: 2026-01-22
Purpose: Predict loan eligibility for demonstration purposes
Intended Use: DEMONSTRATION ONLY - Not for production lending decisions

INTENDED USE:

✓ DO USE FOR:

- Educational demonstrations
- Understanding ML fairness concepts
- Testing and validation workflows
- Fairness auditing techniques

✗ DO NOT USE FOR:

- Actual lending decisions
- Production financial services
- High-stakes decisions affecting individuals
- Autonomous decision-making without human review

CRITICAL LIMITATIONS:

1. BIASED DATA:

- Training data contains historical lending patterns
- Reflects past discrimination and biases
- May perpetuate unfair decisions

2. INCOMPLETE INFORMATION:

- Only uses demographic and income features
- Missing important factors (credit history, employment stability)
- Cannot account for life circumstances

3. MODEL LIMITATIONS:

- Assumes historical patterns predict future outcomes
- Cannot capture economic changes or individual circumstances
- Oversimplifies complex financial decisions

4. FAIRNESS CONCERNS:

- Model may have disparate impact on protected groups